Resumo – Listas

1. A ideia central

Até aqui, cada variável guardava **um valor por vez**. Para ler 5 notas, precisávamos de 5 variáveis; para 50 notas, de 50 variáveis. O código funciona, mas não escala.

Uma **lista** resolve isso: é um tipo de dado que

- armazena **vários valores** em uma única variável;
- mantém uma **ordem** entre os elementos;
- permite **acessar** (por posição) e **percorrer** esses valores.

Macete

Guarde a fórmula do slide: **lista = dados + ordem + posição**. Sempre que o problema tiver "vários valores do mesmo tipo que serão analisados depois", a resposta é uma lista.

2. Criando e acessando

```
notas = [7.5, 8.0, 6.5, 9.0, 5.5]

print(notas[0])
print(notas[2])
print(notas[len(notas) - 1])
print(len(notas))
```

```
7.5
6.5
5.5
5
```

Pense na lista como uma sequência de gavetas numeradas:

índice	0	1	2	3	4
valor	7.5	8.0	6.5	9.0	5.5

Três regras que caem em toda prova:

- os índices **sempre começam em 0**;
- o último índice é **sempre** `len(lista) - 1`;
- acessar um índice que não existe causa erro (`IndexError`).

Atenção

Em uma lista com 5 elementos, `notas[5]` **não existe**. Os índices válidos vão de 0 a 4. O tamanho é 5, mas a última posição é 4.

String e lista são parecidas, mas não iguais

As duas são coleções e as duas podem ser percorridas com `for` e acessadas com índice. A diferença é o que guardam: a string é uma sequência de **caracteres** e representa um texto; a lista possui **elementos independentes** e representa um conjunto de valores.

3. Construindo uma lista durante a execução

Na prática, quase nunca escrevemos os valores à mão. O padrão é: começar com a lista **vazia** e fazer ela crescer com `append`.

```
notas = []

for i in range(5):
    nota = float(input())
    notas.append(nota)
```

Atenção

Em uma lista vazia, `notas[0] = 7.5` dá erro, porque a posição 0 ainda não existe. Para **criar** uma posição nova, use `append`. A atribuição com índice só serve para **alterar** uma posição que já existe.

4. Percorrendo uma lista

Existem duas formas de percorrer, e escolher a certa é metade da questão.

1. Por elemento (for-each): quando só o valor interessa.

```
notas = [7.5, 8.0, 6.5]

for nota in notas:
    print(nota)
```

```
7.5
8.0
6.5
```

2. Por índice: quando a posição também importa.

```
notas = [7.5, 8.0, 6.5]

for i in range(len(notas)):
    print(i, notas[i])
```

```
0 7.5
1 8.0
2 6.5
```

Macete

Pergunte-se: **"eu preciso saber onde o valor está?"** Se a resposta for não, use `for valor in lista`. Se precisar da posição, comparar com o vizinho, alterar a lista ou andar de trás para frente, use `for i in range(len(lista))`.

De trás para frente

O exercício clássico dos slides (ler 5 números e exibir na ordem inversa) tem duas soluções. Com `while`:

```
valores = [1, 2, 3, 4, 5]

i = len(valores) - 1

while i >= 0:
    print(valores[i])
    i -= 1
```

```
5
4
3
2
1
```

Com `for`, usando os três valores do `range`:

```
for i in range(len(valores) - 1, -1, -1):
    print(valores[i])
```

Macete

Leia `range(len(valores) - 1, -1, -1)` assim: **começa** no último índice, **para antes** de -1 (ou seja, o último visitado é o 0) e **anda** de -1 em -1. O segundo valor do `range` nunca é incluído, por isso é -1 e não 0.

5. Funções prontas

```
valores = [12, 7, 25, 3, 18]

print(len(valores))
print(min(valores))
print(max(valores))
print(sum(valores))
```

```
5
3
25
65
```

Atenção

O professor recomenda implementar as suas próprias versões antes de usar as prontas, e as listas de exercícios costumam **proibir** `max`, `min`, `sum` ou `sorted`. Saiba fazer os padrões da próxima seção na mão.

O que pode existir em uma lista?

Números, textos e outros tipos. O Python até permite misturar (`[10, "Ana", 7.5, True]`), mas algumas operações deixam de funcionar: `min` de uma lista com número e texto dá erro. **Boa prática:** mantenha as listas homogêneas, com um único tipo de dado.

6. Os padrões que resolvem quase tudo

Quase toda questão de lista é uma combinação dos padrões abaixo. Reconheça qual é e adapte.

Padrão 1: contador (quantos atendem a uma condição)

```
notas = [7.5, 4.0, 6.5, 9.0, 5.5]
contador = 0

for nota in notas:
    if nota > 6.0:
        contador += 1

print(contador)
```

```
3
```

Receita: percorrer todos, aplicar a condição, acumular.

Padrão 2: acumulador (soma e média)

```
notas = [7.0, 4.0, 6.0, 9.0]
soma = 0

for nota in notas:
    soma += nota

media = soma / len(notas)
print(soma, media)
```

```
26.0 6.5
```

Padrão 3: maior (ou menor) valor

```
notas = [7.5, 4.0, 9.0, 5.5]
maior = notas[0]

for nota in notas:
    if nota > maior:
        maior = nota
```

```
print(maior)
```

```
9.0
```

Macete

Inicialize `maior` com o **primeiro elemento da lista**, nunca com 0. Se todos os valores forem negativos, começar com 0 dá resposta errada. Para o menor, troque `>` por `<`. Se a questão pedir a **posição** do maior, guarde o índice: comece com `pos = 0` e compare `notas[i] > notas[pos]`.

Padrão 4: filtro (construir uma nova lista)

```
valores = [4, -2, 7, 0, -9, 3]
positivos = []

for valor in valores:
    if valor > 0:
        positivos.append(valor)

print(positivos)
```

```
[4, 7, 3]
```

A mesma estrutura serve para **transformar**: troque o `if` por `nova.append(valor * 2)` e você tem "cada valor multiplicado por 2".

Padrão 5: comparar vizinhos

```
valores = [3, 7, 6, 15, 20]

for i in range(1, len(valores)):
    print(valores[i - 1], valores[i], valores[i] - valores[i - 1])
```

```
3 7 4
7 6 -1
6 15 9
15 20 5
```

Macete

Para comparar cada elemento com o **anterior**, comece o `range` em 1 e use `i - 1`. Para comparar com o **próximo**, pare um antes: `range(len(valores) - 1)` e use `i + 1`. Uma lista com n elementos tem $n - 1$ pares de vizinhos.

Padrão 6: verificação (função que retorna True ou False)

```
def todos_positivos(valores):
    for valor in valores:
        if valor <= 0:
            return False

    return True
```

```
print(todos_positivos([3, 8, 1]))
print(todos_positivos([3, -8, 1]))
```

```
True
False
```

Macete

Em perguntas do tipo "**todos** atendem?": procure um **contraexemplo**. Achou um, `return False` **dentro** do laço. O `return True` fica **fora** do laço, depois que todos foram conferidos. Em perguntas do tipo "**existe algum**?", é o contrário: `return True` dentro, `return False` fora.

7. Erros mais comuns

Atenção

- **Passar do limite:** usar `valores[i + 1]` dentro de `range(len(valores))`. Na última volta, `i + 1` não existe.
- **Confundir índice com valor:** em `for i in range(len(v))`, `i` é a posição e `v[i]` é o valor. Em `for x in v`, `x` já é o valor.
- **return True dentro do laço:** a função responde depois de olhar só o primeiro elemento.
- **Média de lista vazia:** dividir por `len(lista)` quando ele vale 0 dá erro de divisão por zero.
- **Esquecer de converter:** `input()` devolve texto. Sem `int()` ou `float()`, a lista fica cheia de strings e as contas não funcionam.
- **Lista alterada dentro de função:** se uma função faz `valores[i] = ...` ou `valores.append(...)` na lista recebida, a lista original do programa principal também muda. Fique atento a isso nas questões de leitura de código.

8. Questões de exemplo resolvidas

Questão 1 — Sequência crescente (desafio dos slides)

Escreva uma função que receba uma lista e retorne `True` se ela está em ordem crescente e `False` caso contrário.

Raciocínio. Entrada: uma lista. Processamento: comparar cada valor com o anterior (padrão 5) e procurar uma quebra (padrão 6). Saída: `True` ou `False`. Se algum valor for menor ou igual ao anterior, a ordem quebrou.

```
def esta_crescente(codigos):
    for i in range(1, len(codigos)):
        if codigos[i] <= codigos[i - 1]:
            return False

    return True
```

```
print(esta_crescente([3, 7, 10, 15, 20]))
print(esta_crescente([3, 7, 6, 15, 20]))
print(esta_crescente([9, 8, 7, 6, 5]))
```

```
True
False
False
```

Questão 2 — Tentativas abaixo da média (desafio dos slides)

Crie a função `contar_abaixo_media` que receba uma lista de tempos e retorne quantas tentativas ficaram abaixo da média.
Raciocínio. São dois padrões em sequência: primeiro o acumulador (para a média) e depois o contador. São **dois laços separados**: não dá para contar antes de conhecer a média.

```
def contar_abaixo_media(tempos):
    soma = 0

    for tempo in tempos:
        soma += tempo

    media = soma / len(tempos)
    contador = 0

    for tempo in tempos:
        if tempo < media:
            contador += 1

    return contador


print(contar_abaixo_media([12.5, 11.8, 13.2, 10.9, 12.0]))
print(contar_abaixo_media([10, 10, 10, 10, 10]))
```

```
3
0
```

No primeiro caso a média é 12.08; ficam abaixo dela 11.8, 10.9 e 12.0. No segundo, nenhum valor é *menor* que a média, pois todos são iguais a ela. (A tabela de casos de teste do slide indica 2 para a primeira lista, mas, fazendo a conta, 12.0 também é menor que 12.08; o resultado correto é 3.)

Questão 3 — Conversor de string (desafio dos slides)

Crie a função `converter_para_lista` que receba uma string e retorne uma lista com cada caractere em uma posição.
Raciocínio. É o padrão 4 sem condição: lista vazia, percorrer, `append`, retornar.

```
def converter_para_lista(texto):
    letras = []
```

```
    for caractere in texto:
        letras.append(caractere)

    return letras


print(converter_para_lista("casa"))
print(converter_para_lista(""))
```

```
['c', 'a', 's', 'a']
[]
```

Questão 4 — Leitura de código

Escreva a saída do programa abaixo.

```
dados = [4, 9, 2, 9, 6]
saldo = 0

for pos in range(len(dados)):
    if pos % 2 == 0:
        saldo = saldo + dados[pos]
    else:
        saldo = saldo - dados[pos]

    print(pos, dados[pos], saldo)

print(max(dados), len(dados))
```

Raciocínio. Monte uma tabela com uma linha por volta do laço. A condição olha para a **posição** (`pos`), e não para o valor: posições pares somam, posições ímpares subtraem.

`pos`	`dados[pos]`	operação	`saldo`
0	4	soma	4
1	9	subtrai	-5
2	2	soma	-3
3	9	subtrai	-12
4	6	soma	-6

```
0 4 4
1 9 -5
2 2 -3
3 9 -12
4 6 -6
9 5
```

9. Cola rápida

Quero...	**Escrevo...**
criar lista vazia	`valores = []`
adicionar no final	`valores.append(x)`
acessar uma posição	`valores[i]`
alterar uma posição	`valores[i] = x`
tamanho	`len(valores)`
último elemento	`valores[len(valores) - 1]`
percorrer valores	`for v in valores:`
percorrer posições	`for i in range(len(valores)):`
percorrer ao contrário	`for i in range(len(valores) - 1, -1, -1):`
comparar com o anterior	`for i in range(1, len(valores)):`
menor, maior, soma	`min(valores)`, `max(valores)`, `sum(valores)`